# Reinforcement Learning

An Introduction

# Course Content

1. Introduction to Reinforcement Learning
2. Markov Decision Processes (MDPs)
3. Policy and Value Functions
4. Dynamic Programming (DP) for RL
5. Model-Free methods
6. Value Function Approximation
7. **Policy-Gradient and Actor-Critic Methods**
8. Deep RL
9. TP Project

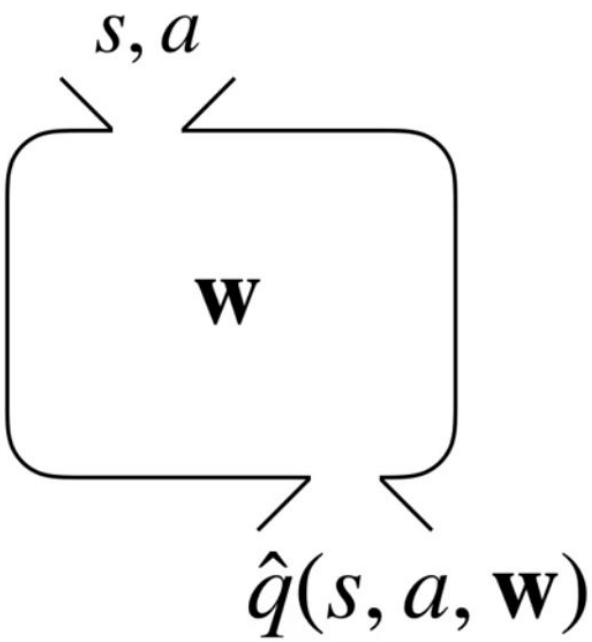
# Policy-Gradient and Actor-Critic Methods

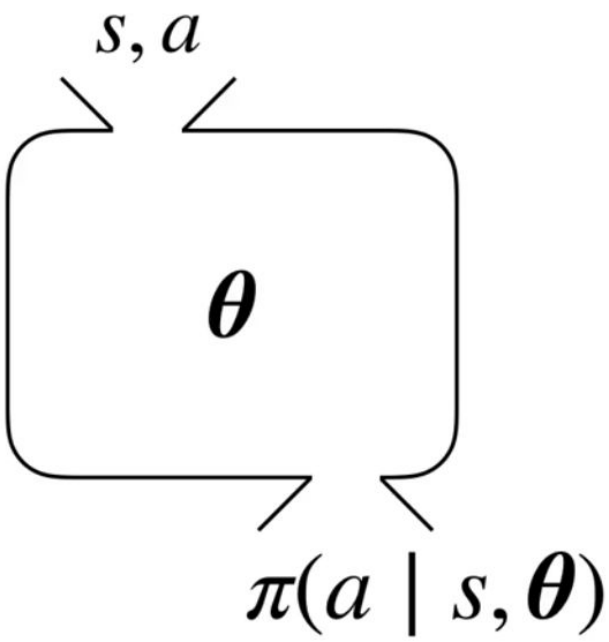
# Introduction

Recall

- **Value-based:**
  - No Policy
  - Value Function
- **Policy-based:**
  - Policy
  - No Value Function
- **Actor-Critic:**
  - Policy
  - Value Function
- **Model-free:**
  - Policy and/or Value Function
  - No Model
- **Model-based:**
  - Policy and/or Value Function
  - Model

# Direct Policy Learning



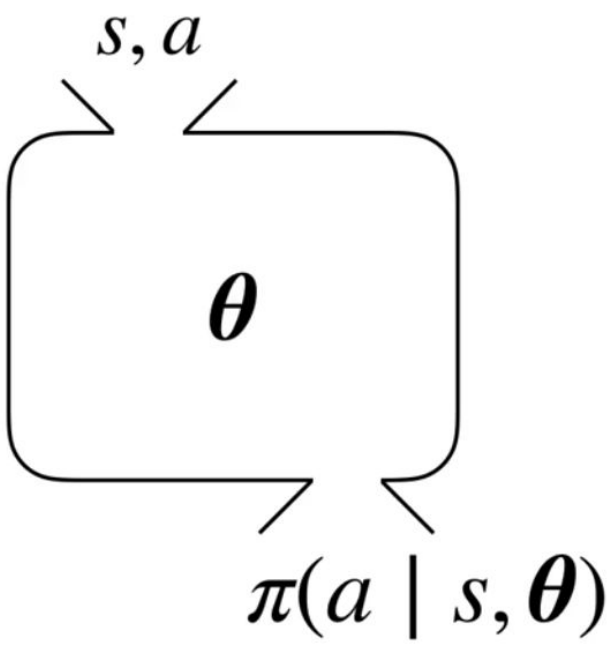



# Policy Parameterization

Limitations

$\pi(a \mid s, \boldsymbol{\theta}) \geq 0$ **for all** $a \in \mathscr{A}$ **and** $s \in \mathscr{S}$

$\sum_{a \in \mathscr{A}} \pi(a \mid s, \boldsymbol{\theta}) = 1$ **for all** $s \in \mathscr{S}$

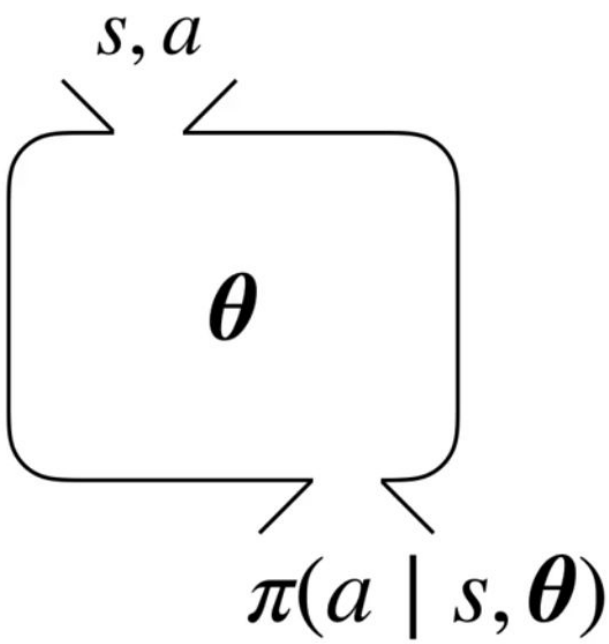


# Policy Parameterization

Softmax Policy

**Action Preference**

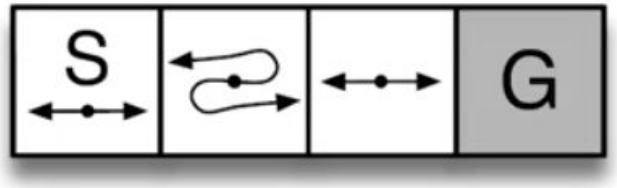
$$\pi(a \mid s, \boldsymbol{\theta}) \doteq \frac{e^{\boxed{h(s,a,\boldsymbol{\theta})}}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\boldsymbol{\theta})}}$$



# Action Preferences vs Action Values

# Advantages of Policy Parameterization

- They can independently reduce exploration as time progresses.
- They can prevent failures caused by deterministic policies when using limited function approximation.
- In some cases, the policy can be simpler than the value function




$R = -1$

- **With deterministic policy: *V(S) = -∞*** for any action to left or right
- **With stochastic policy: *V(S) = -11.6*** to the right

# Policy Parameterization

**Advantages:**

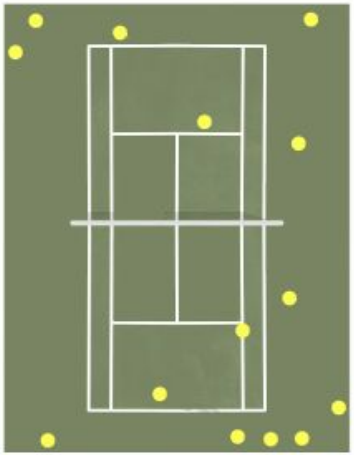
- Improved convergence characteristics.
- Well-suited for high-dimensional or continuous action spaces.
- Capable of learning stochastic policies.

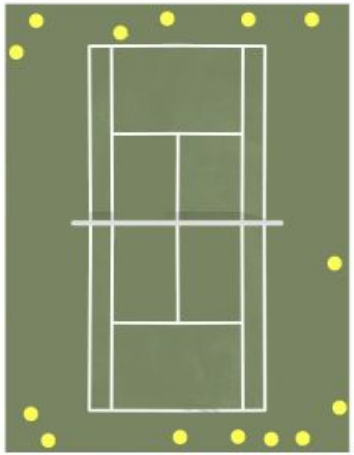
**Disadvantages:**

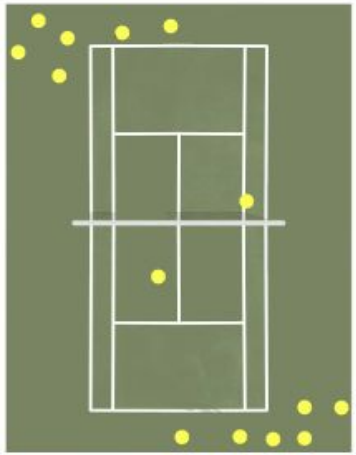
- Often converges to a local optimum instead of the global one.
- Policy evaluation is generally inefficient and prone to high variance.

# General intuitions

Example: a (cheap) tennis ball collector



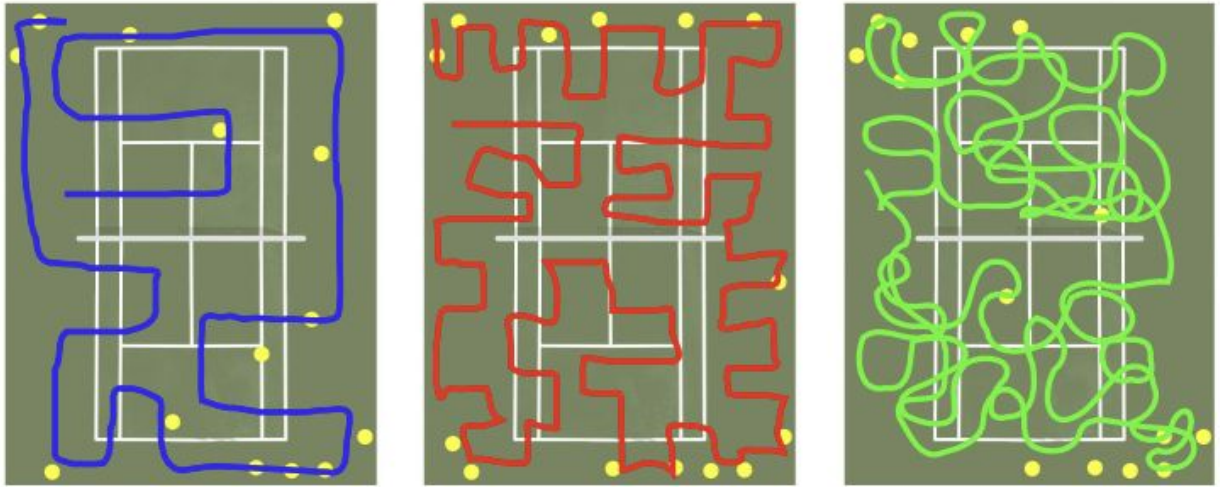




- A robot without a ball sensor
- Travels on a tennis court based on a parametrized controller
- Performance (R): number of balls collected in a given time
- Just depends on robot trajectories and ball positions

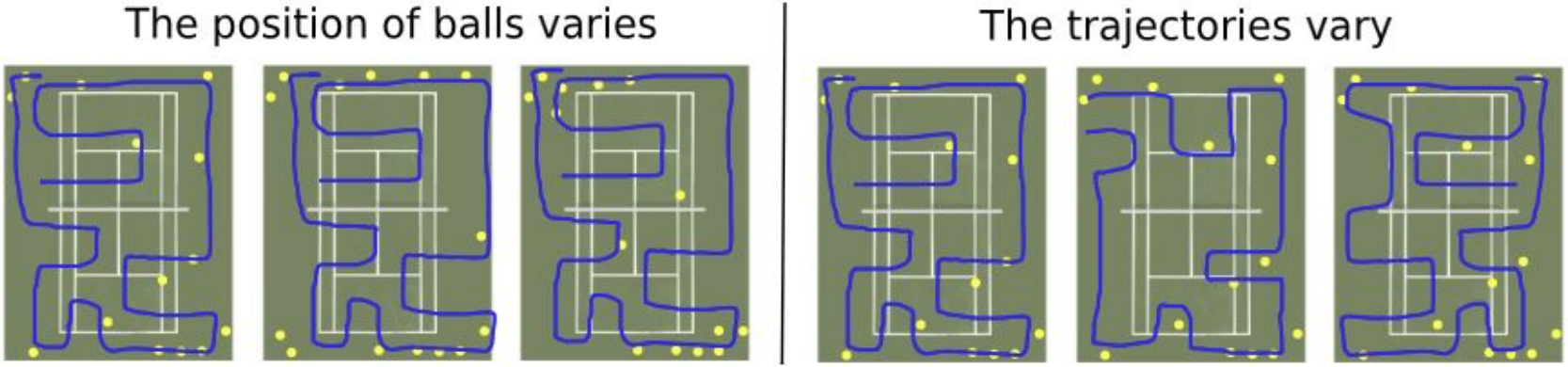
# General intuitions

Possible Policies



- Controller parameters: proba of turn per time step, travelling speed
- How do the parameters influence the performance?
- **Policy search**: find the optimal policy parameters

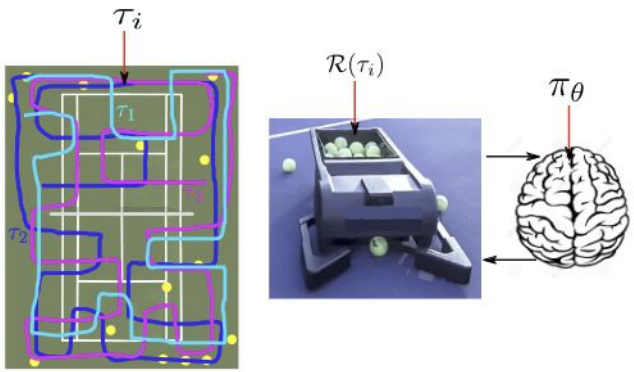
# Two sources of stochasticity




- From the environment: position of the balls
- From the policy, if it is stochastic
- The performance can vary a lot → need to repeat trials
- Tuning parameters can be hard

# Policy Search

# Policy Search - Formalization




- $\tau_i$ a robot trajectory
- $\pi_{\boldsymbol{\theta}}$ the parametrized policy of the robot
- $R(\tau_i)$ the corresponding **return**
  - Return is NOT (exactly) the Gain

$$R(\tau_i) = \sum_{k=0}^{T} r_k = G_0^{\gamma=1}$$

We want to optimize $\boxed{J(\boldsymbol{\theta}) = \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\theta}}}[R(\tau)]}$, the **global utility function**

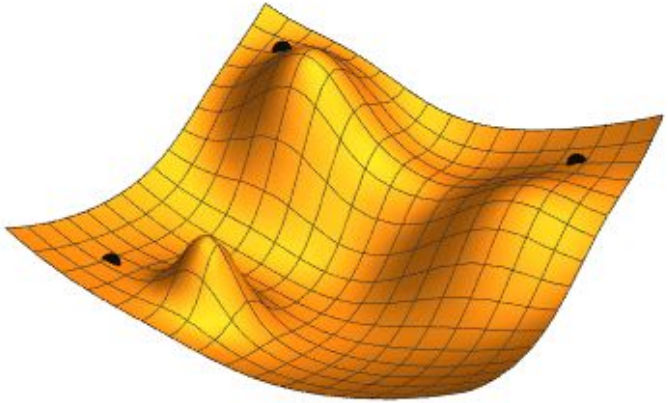
We tune policy parameters θ, thus the goal is to find :

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmax}}\, J(\boldsymbol{\theta}) = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \sum_{\tau} P(\tau|\boldsymbol{\theta}) R(\tau)$$

where $P(\tau|\boldsymbol{\theta})$ is the probability of trajectory τ under policy $\pi_{\boldsymbol{\theta}}$

# Policy Gradients approach

- General idea: increase $P(\tau|\boldsymbol{\theta})$ for trajectories $\tau$ with a high return *J(θ).*
- The best way to improve *J(θ)* is through gradient ascent over *θ*:

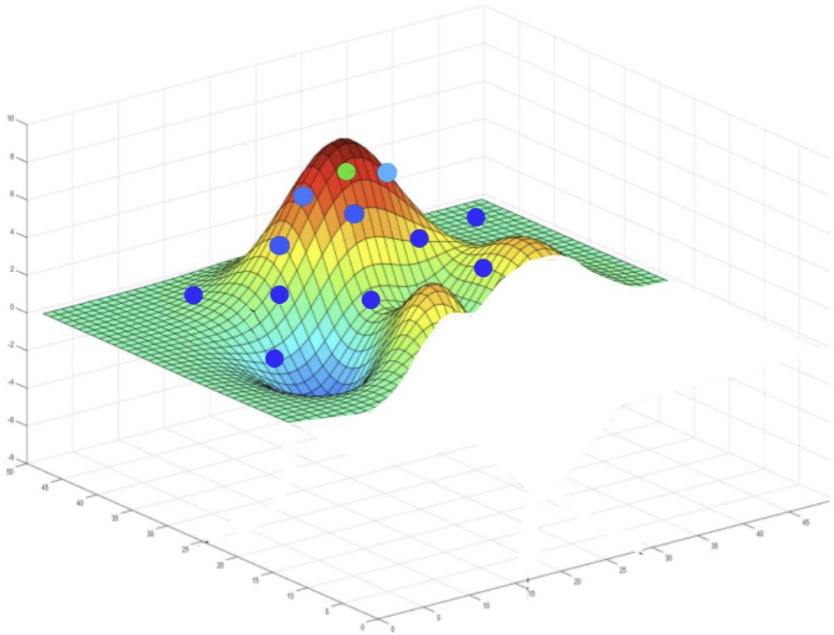


- Thus we can apply $\boldsymbol{\theta}' = \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} \mathbf{J}(\boldsymbol{\theta})$ from analytical knowledge of ***J(θ)***
- Issue: in general, the function ***J(θ)*** is unknown
- How can we apply gradient ascent without knowing the function?
- The answer is the **Policy Gradient Theorem**

# Direct Policy Search

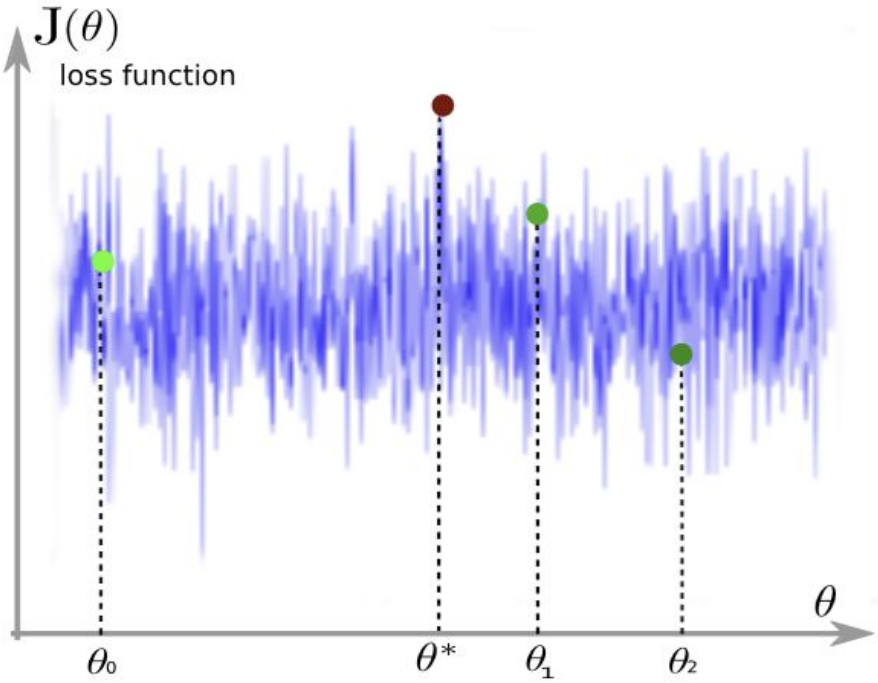
# Random search ?

Black Box optimization



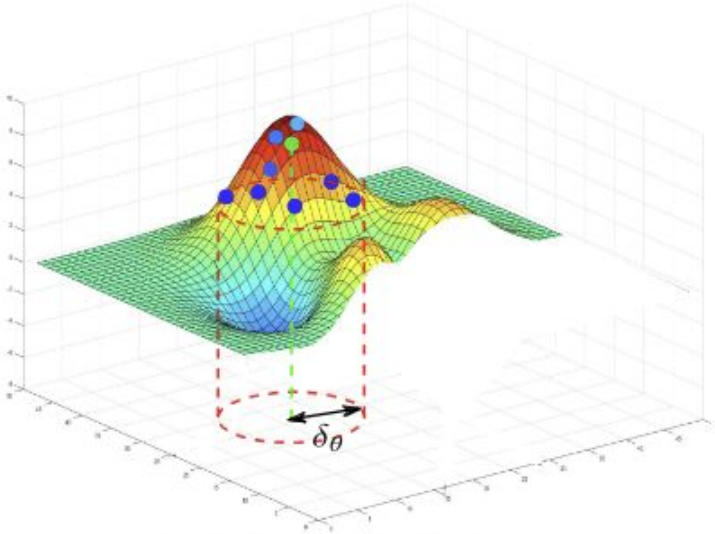
- Select random $\boldsymbol{\theta}$
- *Generate trajectories with* $\pi_{\boldsymbol{\theta}}$
- Evaluate performance = $\mathbf{J}(\boldsymbol{\theta})$
- Keep the best one ..

# Random search - Not so good




- At least its simple..
- “Blind” Algorithm
- The space of possible policies is too big !
- We can do better if **J is smooth**

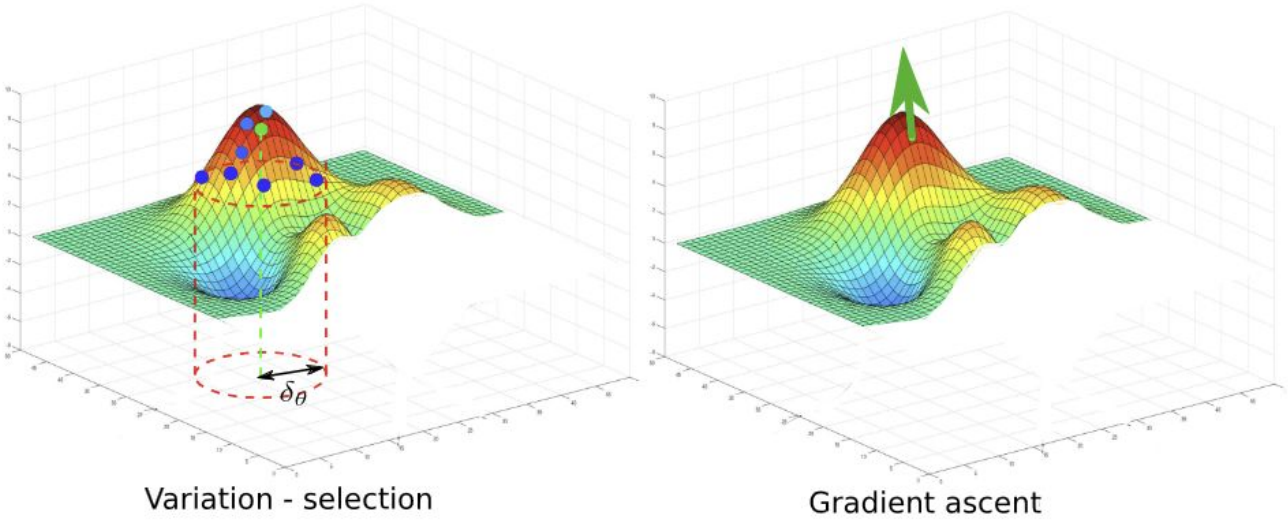
# Meta heuristics




Variation - selection

- More general than RL
- All “sample and test” methods “Smarter” than random sampling
- Under **Locality assumption:** *The function is locally smooth, good solutions are close to each other*
  - Genetic algorithms
  - Simulated Annealing
  - Ant colony algorithms

# What if get the direction uphills ?




- **Gradient ascent**: Following the gradient from analytical knowledge
- **Issue:** in general, the function ***J(θ)*** is unknown 😢
- How can we apply gradient ascent without knowing the function?
    - The answer is the **Policy Gradient Theorem**

# Policy Gradients Theorem

# Policy Gradients

We are looking for $\boldsymbol{\theta}^* = \text{argmax}_{\boldsymbol{\theta}}\, J(\boldsymbol{\theta}) = \text{argmax}_{\boldsymbol{\theta}} \sum_\tau P(\tau|\boldsymbol{\theta})R(\tau)$

$$
\begin{aligned}
\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) &= \nabla_{\boldsymbol{\theta}} \sum_\tau P(\tau|\boldsymbol{\theta})R(\tau) \\
&= \sum_\tau \nabla_{\boldsymbol{\theta}} P(\tau|\boldsymbol{\theta})R(\tau) && \text{* gradient of sum is sum of gradients} \\
&= \sum_\tau \frac{P(\tau|\boldsymbol{\theta})}{P(\tau|\boldsymbol{\theta})} \nabla_{\boldsymbol{\theta}} P(\tau|\boldsymbol{\theta})R(\tau) && \text{* Multiply by one} \\
&= \sum_\tau P(\tau|\boldsymbol{\theta}) \frac{\nabla_{\boldsymbol{\theta}} P(\tau|\boldsymbol{\theta})}{P(\tau|\boldsymbol{\theta})} R(\tau) && \text{* Move one term} \\
&= \sum_\tau P(\tau|\boldsymbol{\theta}) \nabla_{\boldsymbol{\theta}} \log P(\tau|\boldsymbol{\theta})R(\tau) && \text{* by property of gradient of log} \\
&= \mathbb{E}_\tau[\nabla_{\boldsymbol{\theta}} \log P(\tau|\boldsymbol{\theta})R(\tau)] && \text{* by definition of the expectation}
\end{aligned}
$$

- We want to compute $\mathbb{E}_\tau[\nabla_{\boldsymbol{\theta}} \log P(\tau|\boldsymbol{\theta}) R(\tau)]$
- We do not have an analytical expression for $P(\tau|\boldsymbol{\theta})$
- Thus the gradient $\nabla_{\boldsymbol{\theta}} \log P(\tau|\boldsymbol{\theta}) R(\tau)$ cannot be computed
- Let us reformulate $P(\tau|\boldsymbol{\theta})$ using the policy $\pi_{\boldsymbol{\theta}}$
- What is the probability of a trajectory?
- At each step, probability of taking each action (defined by the policy) times probability of reaching the next state given the action
- Then product over states for the whole horizon $H$

$$P(\tau|\boldsymbol{\theta}) = \prod_{t=1}^{H} p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t).\pi_{\boldsymbol{\theta}}(\mathbf{a}_t|\mathbf{s}_t)$$

- (Strong) Markov assumption here: holds if steps are independent

# **Policy Gradients -** *Getting rid of the dynamics*

- Thus, under Markov assumption,

$$
\begin{aligned}
\nabla_{\boldsymbol{\theta}} \log \mathrm{P}(\tau|\boldsymbol{\theta}) &= \nabla_{\boldsymbol{\theta}} \log[\prod_{t=1}^{H} p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t).\pi_{\boldsymbol{\theta}}(\mathbf{a}_t|\mathbf{s}_t)] \\
&\quad \text{* log of product is sum of logs} \\
&= \nabla_{\boldsymbol{\theta}} [\sum_{t=1}^{H} \log \mathrm{p}(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) + \sum_{t=1}^{H} \log \pi_{\boldsymbol{\theta}}(\mathbf{a}_t|\mathbf{s}_t)] \\
&= \nabla_{\boldsymbol{\theta}} \sum_{t=1}^{H} \log \pi_{\boldsymbol{\theta}}(\mathbf{a}_t|\mathbf{s}_t) \text{ * because first term independent of } \boldsymbol{\theta} \\
&= \sum_{t=1}^{H} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(\mathbf{a}_t|\mathbf{s}_t) \text{ * no dynamics model required!}
\end{aligned}
$$

- The key is here: we know $\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(\mathbf{a}_t|\mathbf{s}_t)$!

# Policy Gradients - End

- The expectation $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\tau}[\nabla_{\boldsymbol{\theta}} \log P(\tau|\boldsymbol{\theta}) R(\tau)]$ can be rewritten

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\tau}[\sum_{t=1}^{H} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(\mathbf{a}_t|\mathbf{s}_t) R(\tau)]$$

- The expectation can be approximated by sampling over $m$ trajectories:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \approx \frac{1}{m} \sum_{i=1}^{m} \sum_{t=1}^{H} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)}) R(\tau^{(i)}) \tag{4}$$

-

- The policy structure $\pi_{\boldsymbol{\theta}}$ is known, thus the gradient $\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(\mathbf{a}|\mathbf{s})$ can be computed for any pair $(\mathbf{s}, \mathbf{a})$
- We moved from direct policy search on $J(\boldsymbol{\theta})$ to gradient ascent on $\pi_{\boldsymbol{\theta}}$
- Can be turned into a practical (but not so efficient) algorithm

# Policy Gradients

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[R(\tau)\sum_{t=1}^{T}\nabla_\theta \, log \, \pi_\theta(a_t|s_t)\right]$$

$$\approx \quad Loss(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{H}\log\pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})R(\tau^{(i)})$$

- We removed the dependance on the environment !
- Can be turned into a practical algorithm
- At the basis of all **Model-Free methods**

# The Policy Gradient Theorem

Different Gradient Estimations

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \Psi_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

where $\Psi_t$ may be one of the following:

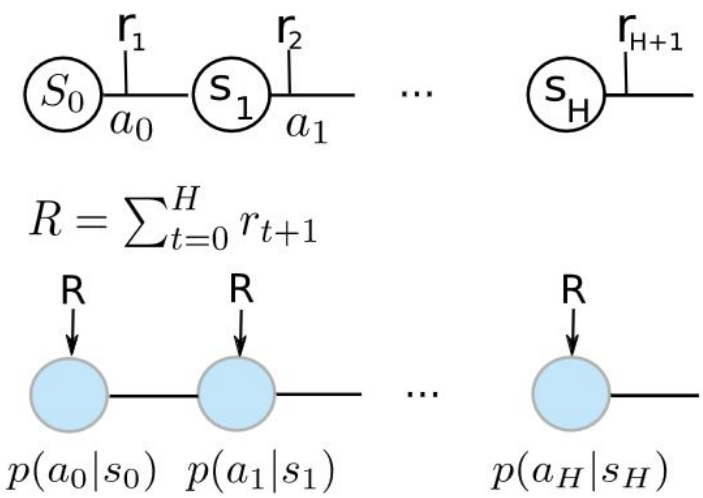
1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory.
2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action $a_t$.
3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula.
4. $Q^\pi(s_t, a_t)$: state-action value function.
5. $A^\pi(s_t, a_t)$: advantage function.
6. $r_t + V^\pi(s_{t+1}) - V^\pi(s_t)$: TD residual.

Fig. 1. A general form of policy gradient methods. (Image source: Schulman et al., 2016)

# Naive Algorithms

*“So thats a wrap babe, we solved RL !”*

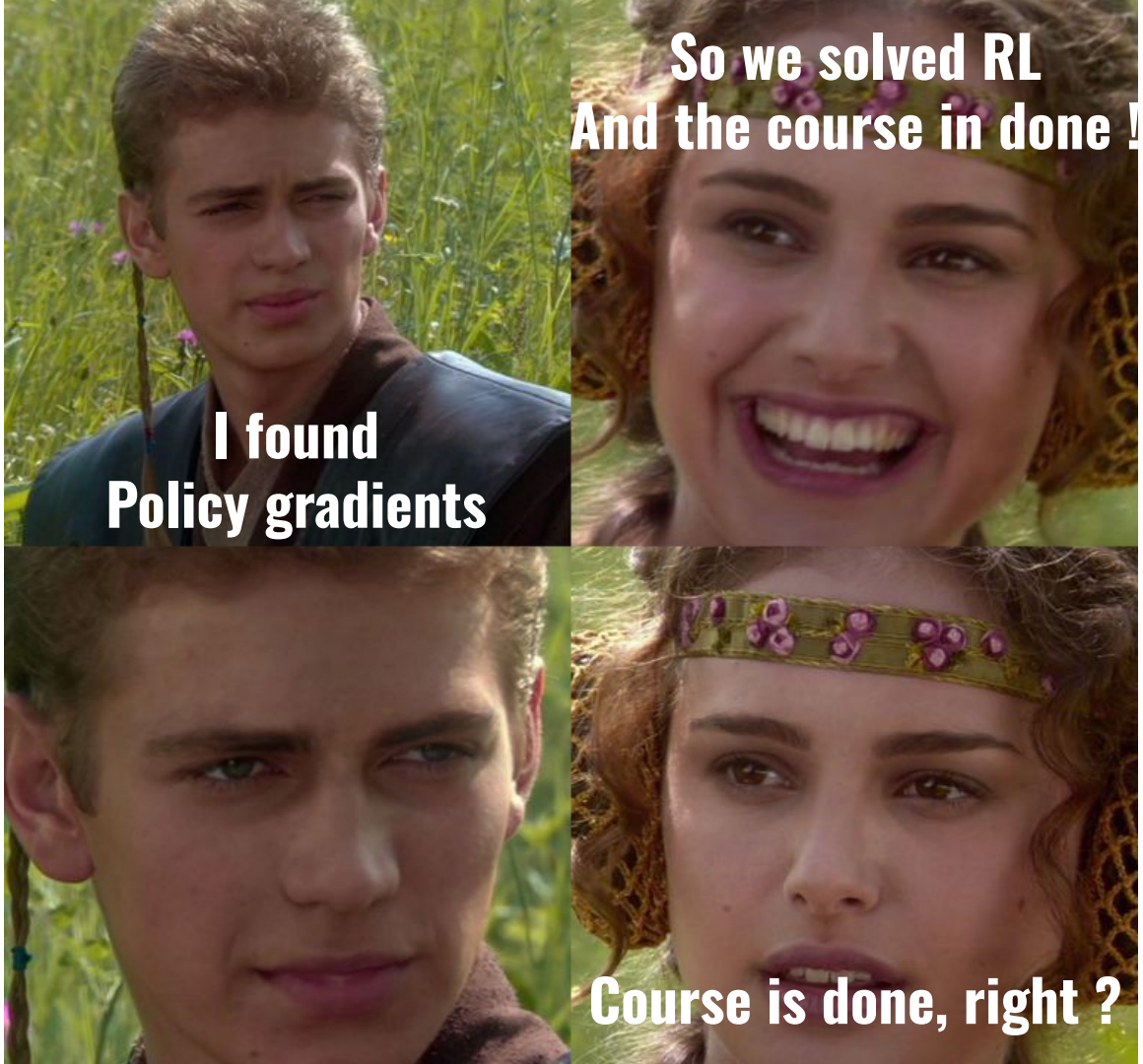
# Naive - Basic Policy Gradients




- Sample a set of trajectories.
- Compute

$$\boxed{Loss(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{H}\log\pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})R(\tau^{(i)})} \approx \nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta}) \approx \frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{H}\nabla_{\boldsymbol{\theta}}\log\pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})R(\tau^{(i)})$$

- Minimize the loss using the NN backprop function with your favorite pytorch or tensorflow optimizer (Adam, RMSProp, SGD...)
- Iterate: sample again, for many time steps, **easy right ?**
- Note: if $R(\tau) = 0$, does nothing….


I found
Policy gradients
So we solved RL
And the course in done !
Course is done, right ?

# The problem with the Naive Algorithm

$$Loss(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{H}\log\pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})R(\tau^{(i)})$$

- Needs a large batch of trajectories or suffers from large variance
- **The sum of rewards is not much informative**
- Computing ***R*** from complete trajectories is not the best we can do
- etc…

**Doesn't work well enough in practice !**

- Now we introduce tricks that allow better:
  - Exploration
  - Stability
  - Convergence
  - …

# 1 - Focus on next rewards

- Computing R from complete trajectories is not the best we can do.

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \sim \frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{H}\nabla_{\boldsymbol{\theta}}\log\pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})R(\tau^{(i)})$$

$$\sim \frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{H}\nabla_{\boldsymbol{\theta}}\log\pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})[\sum_{k=1}^{H} r(\mathbf{s}_k^{(i)},\mathbf{a}_k^{(i)})]$$

* split into two parts

$$\sim \frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{H}\nabla_{\boldsymbol{\theta}}\log\pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})[\sum_{k=1}^{t-1} r(\mathbf{s}_k^{(i)},\mathbf{a}_k^{(i)}) + \sum_{k=t}^{H} r(\mathbf{s}_k^{(i)},\mathbf{a}_k^{(i)})]$$

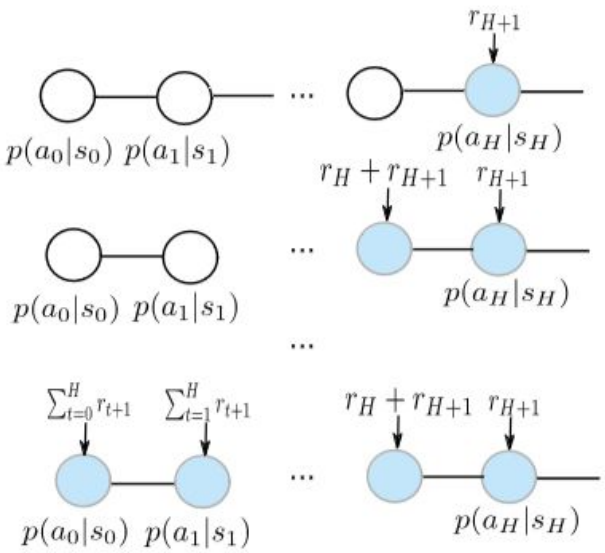
* past rewards do not depend on the current action

$$\sim \frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{H}\nabla_{\boldsymbol{\theta}}\log\pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})[\sum_{k=t}^{H} r(\mathbf{s}_k^{(i)},\mathbf{a}_k^{(i)})]$$

# 1 - Focus on next rewards

First improvement

- Computing R from complete trajectories is not the best we can do.




- Same as Naive Algo
- But the sum is incomplete, and computed backwards
- Slightly less variance, because it ignores irrelevant rewards

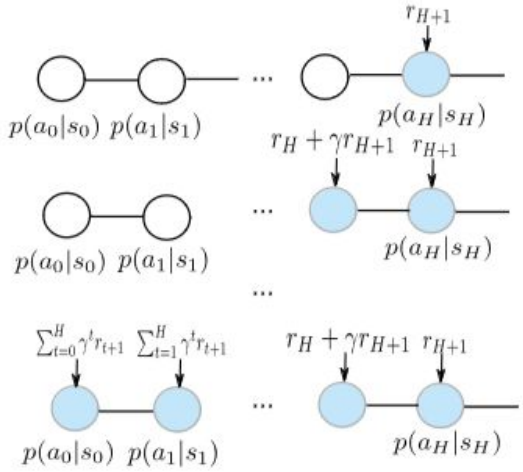
# 2 - Discounting rewards

Second improvement

$$\nabla_\theta J(\theta) \sim \frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{H}\nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})[\sum_{k=t}^{H} r(\mathbf{s}_k^{(i)}, \mathbf{a}_k^{(i)})]$$

* reduce the variance by discounting the rewards along the trajectory

$$\sim \frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{H}\nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})[\sum_{k=t}^{H} \gamma^{k-t} r(\mathbf{s}_k^{(i)}, \mathbf{a}_k^{(i)})]$$



# Monte-Carlo Policy Gradient (REINFORCE)

# Monte-Carlo Policy Gradient (REINFORCE)

$$\begin{aligned}\nabla_\theta J(\theta) &= \mathbb{E}_\pi[Q^\pi(s,a)\nabla_\theta \ln \pi_\theta(a|s)] \\ &= \mathbb{E}_\pi[G_t \nabla_\theta \ln \pi_\theta(A_t|S_t)] \qquad \text{; Because } Q^\pi(S_t,A_t)=\mathbb{E}_\pi[G_t|S_t,A_t]\end{aligned}$$

- Adjust the parameters using **stochastic gradient ascent**,
- Utilize the return ***G(t)*** as an unbiased estimate of $Q_{\pi_\theta}(s_t, a_t)$

- The parameter update rule is given by:

$$\theta \leftarrow \theta + \alpha\gamma^t G_t \nabla_\theta \ln \pi_\theta(A_t|S_t)$$

# Monte-Carlo Policy Gradient (REINFORCE)

**REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for $\pi_*$**

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\qquad (G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

# Monte-Carlo Policy Gradient (REINFORCE)

- Update parameters by stochastic gradient ascent
- Using policy gradient theorem
- Using return $R_T$ as an *unbiased* sample of $Q^{\pi_\theta}(s_t, a_t)$

```
function REINFORCE
    Initialise θ arbitrarily
    for each episode {s_1, a_1, r_2, ..., s_{T-1}, a_{T-1}, r_T} ~ π_θ do
        for t = 1 to T − 1 do
            θ ← θ + α∇_θ log π_θ(s_t, a_t) G_t
        end for
    end for
    return θ
end function
```

-> **get one estimate by timesteps = more information than Naive algorithm**

# A2C, A3C

# Another trick - Adding a baseline

## Reducing Variance with a Baseline

- Reminder:

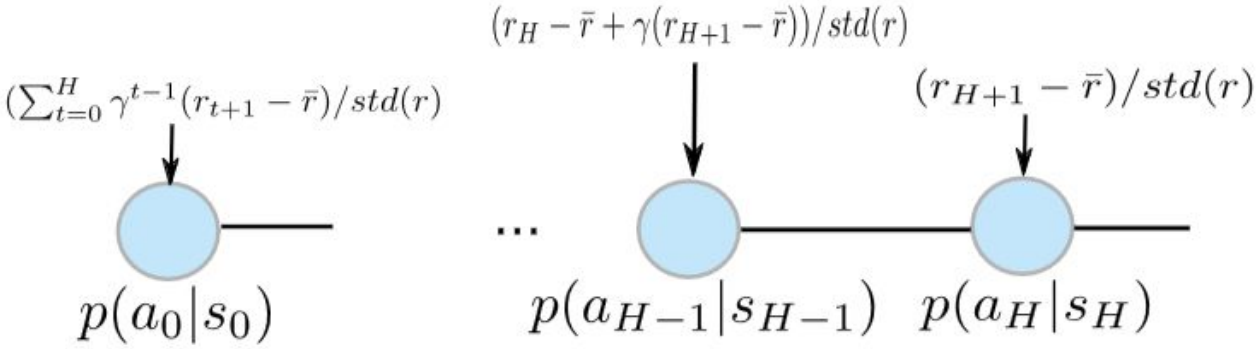
$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} \sum_{t=1}^{H} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) [\sum_{k=t}^{H} \gamma^{k-t} r(\mathbf{s}_k^{(i)}, \mathbf{a}_k^{(i)})]$$

- If all rewards are positive, the gradient increases all probabilities
- But with renormalization, only the largest increases emerge
- We can substract a "baseline" to (6) without changing its mean:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} \sum_{t=1}^{H} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) [\sum_{k=t}^{H} \gamma^{k-t} r(\mathbf{s}_k^{(i)}, \mathbf{a}_k^{(i)}) - b]$$

- A first baseline is the average return $\bar{r}$ over all states of the batch
- Intuition: returns greater than average get positive, smaller get negative
- Use $(r_{t+1}^{(i)} - \bar{r})$ and divide by std $\rightarrow$ get a mean $= 0$ and a std $= 1$
- This improves variance (does the job of renormalization)

# 3 - Adding a baseline




- Estimate *mean(**r**)* and *std(**r**)* from all rollouts
- Same as Algorithm 2, using normalized **r :** $(r_{t+1}^{(i)} - \bar{r})/std(r)$
- Suffers from even less variance
- **Does not work if all rewards r are identical (e.g. CartPole)**

# 3 - Adding a baseline

Reducing Variance with a Baseline

- We subtract a baseline function B(s) from the policy gradient
- This can reduce variance, without changing expectation

$$\begin{aligned}\mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a) B(s)\right] &= \sum_{s\in\mathcal{S}} d^{\pi_\theta}(s) \sum_a \nabla_\theta \pi_\theta(s,a) B(s) \\ &= \sum_{s\in\mathcal{S}} d^{\pi_\theta} B(s) \nabla_\theta \sum_{a\in\mathcal{A}} \pi_\theta(s,a) \\ &= 0\end{aligned}$$

- A good baseline is the state value function
- So we can rewrite the policy gradient using the **advantage function**

$$A^{\pi_\theta}(s,a) = Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s)$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ A^{\pi_\theta}(s,a)\right]$$

*Note: detailed computation can be found here*

# The Advantage function

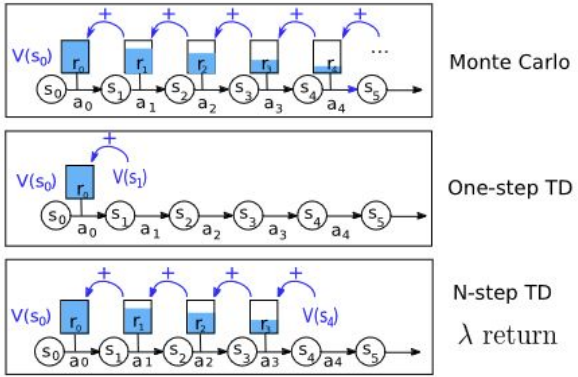
- The advantage function, ***A(s,a)*** , measures the advantage of taking action ***a*** in state ***s*** over the expected value of the state ***V(s)*** under the current policy.

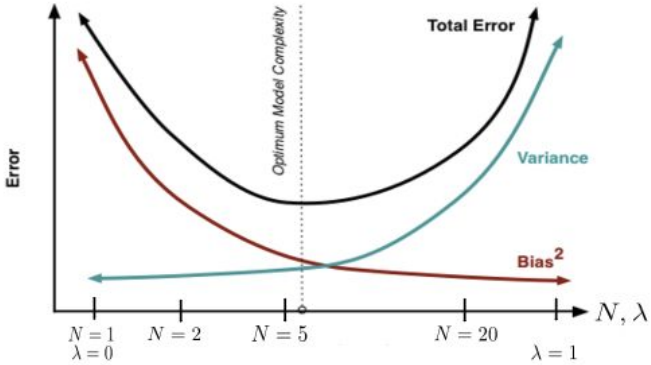
$$A(s,a) = Q(s,a) - V(s)$$

The advantage function, then, provides a measure of how **much better or worse an action is compared to the average action.**

# Bias-variance compromise

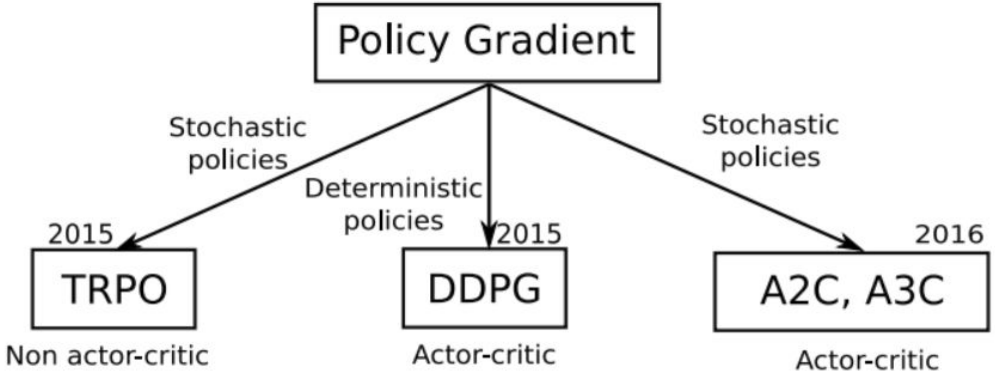
How can we estimate the advantage ?



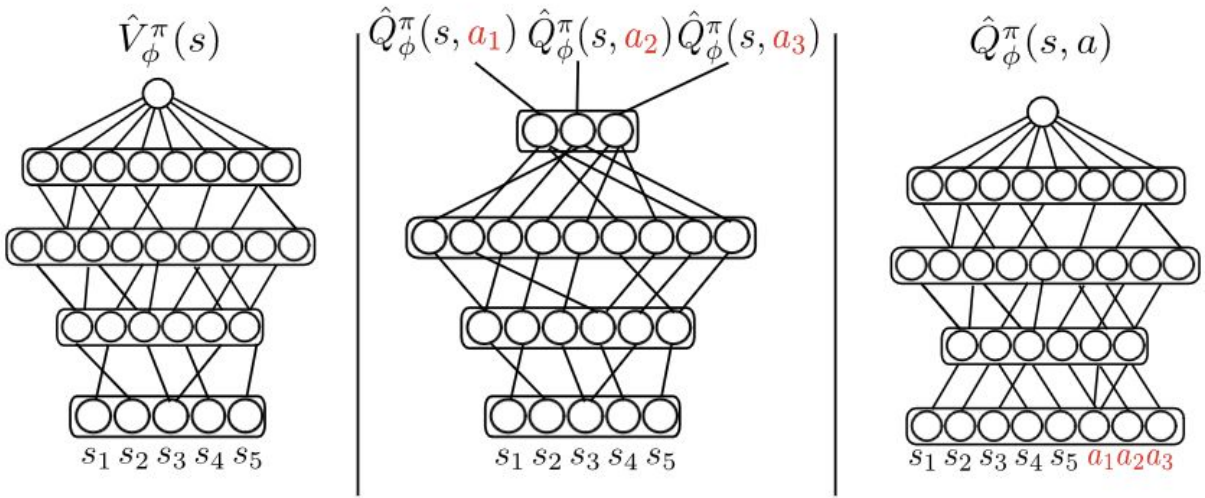

- **Monte Carlo**: unbiased estimate of the critic
  - But MC suffers from variance due to exploration (+ stochastic trajectories)
  - MC on-policy → no replay buffer → less sample efficient
- **Bootstrap** (e.g TD: **SARSA)** is sample efficient but suffers from bias and is unstable
  - **N-step TD** or **λ return**: control the bias-variance compromise
  - Acts on critic, indirect effect on performance

# Advantage Actor-Critic (A2C)



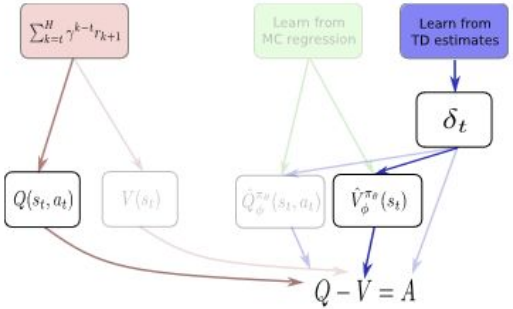

- A crucial move from **Policy Gradient** methods to **Actor-Critic** methods
- The earliest actor-critic algorithm of the deep RL era using stochastic policies
- It directly derives from the basic **Policy Gradient** method
- The critic is learned using bootstrap, which makes it an actor-critic algorithm

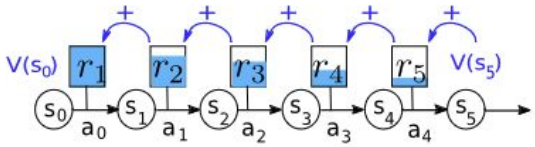
# Choice of a Critic




- Note that **a2c is Actor-Critic, but on-policy**, so one cannot equate Actor-Critic and off-policy

# Computation of Advantage




- To perform policy gradient updates, one needs to compute
- By definition, ***A(st, at) = Q(st, at) – V (st)***
- A2C computes the advantage with $\boxed{\hat{A}_\phi(\mathbf{s}_t, \mathbf{a}_t) = R_t(\mathbf{s}_t) - V_\phi(\mathbf{s}_t)}$
- $R_t(\mathbf{s}_t) = \sum_{i=0}^{N-1} \gamma^i r_{t+i+1} + \gamma^N V_\phi(\mathbf{s}_{t+N})$ is the return of the current N-step trajectory from state ***st***
- **Rt(st)** can be seen as an approximate of ***Q(st, at)*** computed along one Trajectory
- Actually, computed on N steps rather than the full trajectory

# Summary - distinctions

- To perform policy gradient, you need the advantage function
- Computes the advantage function as value function minus the return of the current N-step trajectory
- Adds entropy regularization to favor exploration in the gradient calculation step
- Uses N-step updates
- Does not use a replay buffer
- Note that **a2c is Actor-Critic, but on-policy**, so one cannot equate Actor-Critic and off-policy

# Wrap Up

# The Right distinction

- **Off-policy** versus **On-policy** is *not so clear*, as being off-policy is a matter of degree.
- Being actor-critic, using a replay buffer does not lead to a clear-cut distinction (a2c blurs the classification)
- The right distinction is between value-based approaches (start from a critic) and policy-based approaches (start from the gradient on the policy)
  - `a2c, dqn, ddpg, td3, sac, tqc, droq`are **value-based**
  - `REINFORCE, trpo, acktr, ppo`are **policy-based**
- Hint: for a value-based algo, try a larger critic, for a policy-based algo, try a larger policy

# TP

# Practice Lab: Softmax Actor-Critic - Pendulum Swing-Up

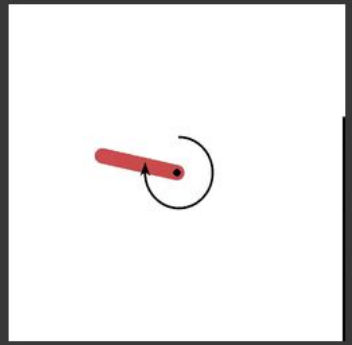
## Practice Lab: Softmax Actor-Critic - Pendulum Swing-Up

**In this assignment, you will:**

1. Develop a softmax-based Actor-Critic (AC) agent for a **continuing** task, utilizing the **average reward** framework.
2. Learn how to represent the policy as a parameterized function for discrete action spaces.
3. Explore methods to approximate and sample the gradient of the objective for updating the actor.
4. Understand the process of updating the critic using the differential temporal difference (TD) error.

## Pendulum Swing-Up Environment

This project is designed to implement the **Pendulum-v0** environment in OpenAI Gym using **policy-gradient** and **average reward framework**. The code is broken down into various sections to make it easier to understand how Softmax Actor-Critic with average reward works.



# References

[1] David Silver, Lectures on Reinforcement Learning, 2015

[2] Reinforcement Learning and Advanced Deep Learning (Sorbonne) - Olivier Sigaud

[3] Sutton, R. S. and Barto, A. G. (2018), Reinforcement Learning: An Introduction (Second edition). MIT Press

Olivier Sigaud Youtube Channel

[5] Comprehensive post on Policy Gradients by Lilian Weng